

AMRITA VISHWA VIDYAPEETHAM

22AIE458 — Mobile Application Development

PROJECT REPORT

NEXORA

Campus Placement Recruitment System

An Android application with explainable eligibility, match scoring, readiness analytics and a Career Action Engine

Submitted by

Name	Roll Number
Mandala Ruthik Reddy	AM.SC.U4AIE23129
Hasthi Naresh	AM.SC.U4AIE23113

Under the guidance of

Rahul Varma U

Faculty Guide

Project Description and Details

Abstract

NEXORA is a native Android application, written in Java with XML layouts, that digitises a college's campus placement process in a single APK with two role-based portals. Students maintain a profile with an officer-verified academic record, discover placement drives, see exactly why they are or are not eligible for each drive, receive an explainable match score, apply with server-enforced duplicate prevention, follow each application through a recruitment timeline and log preparation activities that feed a placement-readiness dashboard. A deterministic Career Action Engine combines these results into a short, ranked plan of next steps on the student home screen. Placement officers verify academic records, manage companies and drives, advance or reject applicants stage by stage, publish announcements and view aggregate analytics, including a privacy-preserving preparation-insights summary. The backend uses Firebase Authentication and Cloud Firestore on the free Spark plan; all privileged rules are enforced by Firestore Security Rules, and Room provides offline access. The application is deployed on a live Firebase project and was verified end to end with the officer portal on an Android emulator and the student portal on a physical phone. It is also verified by 65 JVM unit tests (99.2% line coverage of the domain package), 48 Firestore Security Rules tests on the Firebase Emulator Suite and 6 instrumented tests on an Android 15 emulator.

Problem Statement

Campus placement is commonly coordinated through spreadsheets, notice boards and messaging groups. Students cannot easily tell which drives they qualify for or why, they apply to unsuitable roles, they miss deadlines and interview rounds, and they receive little guidance about what to prepare. Placement officers spend time checking eligibility manually, have no reliable record of academic data, and lack an overview of applicant progress and preparation. Because self-reported academic data can be edited freely, any eligibility check built on it is untrustworthy unless the data is verified and the rules are enforced on the server.

Objectives

1. Provide separate, role-based student and placement-officer portals in one Android application.
2. Make eligibility explainable: evaluate CGPA, branch, active backlogs and graduation year separately and explain each outcome.
3. Rank opportunities with a transparent, documented match score rather than an opaque recommendation.
4. Track preparation activity and compute a placement-readiness score with actionable suggestions.
5. Connect eligibility, ranking, readiness and the recruitment journey through a rule-based Career Action Engine.

6. Enforce trust-critical rules (roles, verification, eligibility, deadlines, duplicates, outcomes) on the server with Firestore Security Rules.
7. Keep the application usable offline and keep the project within the free Firebase Spark plan.

Proposed Solution

The solution separates three concerns. A pure-Java domain layer implements the business logic (eligibility, match score, readiness, workflow, analytics and the Career Action Engine) so that it can be unit-tested on the JVM. A data layer of repositories hides Firebase and Room behind LiveData, using lifecycle-aware snapshot listeners and a Room write-through cache. A UI layer of activities and fragments observes two ViewModels that recompute every derived result whenever the underlying data changes. Academic fields become read-only once a placement officer verifies them, and every write is checked by Firestore Security Rules, which re-evaluate eligibility and reject duplicate or forged applications independently of the client.

Target Users

- **Students:** final- and pre-final-year students who browse drives, apply and prepare for recruitment rounds.
- **Placement officers:** staff of the placement office who verify records, run drives, decide outcomes and monitor progress. Officers create their account in the app by choosing "Placement officer" at sign-up (an open sign-up chosen for this academic project; see Section 4.5).

Technology Stack

Layer	Technology (version as configured in the project)
Language and UI	Java 17 source/target; XML layouts; Material Components for Android 1.14.0 (Material 3 theme, light and dark)
Build	Android Studio, Gradle 9.4.1 with version catalog, Android Gradle Plugin 9.2.1; minSdk 26, compile/target SDK 36
Architecture	MVVM + Repository; AndroidX Lifecycle 2.10.0 (ViewModel, LiveData); Fragment 1.8.9; manual dependency container (AppContainer)
Backend	Firebase BoM 34.19.0: Firebase Authentication (email/password), Cloud Firestore, Firebase Cloud Messaging (optional)
Local storage and background work	Room 2.8.5 (SQLite), WorkManager 2.11.0
Security	Firestore Security Rules (rules_version 2) tested with @firebase/rules-unit-testing on the Firebase Emulator Suite
Testing	JUnit 4, JaCoCo coverage, AndroidX Test, Espresso, Room testing, Node.js test runner

Table 1: Technology stack

System Architecture

NEXORA follows the MVVM + Repository pattern recommended in the Android architecture guide. The UI layer observes LiveData exposed by StudentViewModel and OfficerViewModel; the ViewModels call repositories for data and pure-Java domain engines for every calculation; repositories wrap Cloud Firestore, Firebase Authentication, Room and WorkManager. Figure 1 in Chapter 3 shows the layers and their dependencies.

Project Scope

In scope (implemented):

- Student and officer portals, onboarding, registration as a student or placement officer, login, logout and password reset.
- Officer verification of academic records; company and drive management with criteria, required skills, preparation focus, deadlines and recruitment stages.
- Explainable eligibility, explainable match score, applications with duplicate prevention, recruitment timeline and officer stage decisions.
- Readiness tracking, Career Action Engine, announcements, automatic new-drive notifications, local notifications, officer analytics and preparation insights.
- Offline browsing of cached drives and applications, offline bookmarks and offline activity logging.

Out of scope: resume file upload (resumes are external links because Firebase Storage needs the paid plan), server-sent push notifications (no Cloud Functions on the Spark plan), recruiter accounts, hiring-outcome prediction and a web portal.

Table of Contents

Project Description and Details 2
Abstract 2
Problem Statement 2
Objectives 2
Proposed Solution 3
Target Users 3
Technology Stack 3
System Architecture 4
Project Scope 4
1. Features Implemented 7
1.1 Student Portal 7
1.2 Placement Officer Portal 7
1.3 Explainable Eligibility Engine 8
1.4 Explainable Match Score 8
1.5 Placement Readiness Dashboard 8
1.6 Career Action Engine 8
1.7 Application Workflow 9
1.8 Offline Behaviour 9
1.9 Announcements, Notifications and Analytics 9
2. Application Screens 11
3. System Design and Blueprints 12
3.1 System Architecture 12
3.2 Module Structure 13
3.3 Use Cases 14
3.4 Integrated Workflow 15
3.5 Screen Navigation 16
3.6 Application Lifecycle 17
3.7 Data Model and Security 18
4. Implementation and Code 19
4.1 Key Java Classes 19
4.2 XML Layouts and Resources 19
4.3 Firebase Integration 19
4.4 Code Excerpts 20
4.5 Firestore Security Rules 21
5. Testing and Results 23
5.1 Automated Testing 23

5.2 Manual Testing 23
5.3 Limitations of Testing 24
6. Output Screenshots 25
6.1 Student Portal 25
6.2 Placement Officer Portal 28
6.3 Live Deployment 30
7. Conclusion and Future Scope 31
7.1 Conclusion 31
7.2 Future Scope 31
8. References 32
9. Appendix 33
A. Build, Run and Test Commands 33
B. Source File Inventory 33
C. Configuration and Security Notes 33

1. Features Implemented

All features listed below were implemented in the repository and exercised on the Android emulator against the Firebase Emulator Suite, and then on the live Firebase project with a physical phone. Figures referenced in this chapter appear in Chapter 6.

1.1 Student Portal

- **Onboarding and authentication:** three-slide onboarding, sign-in, registration with a Student / Placement officer choice, password reset and logout. Skills typed as a comma-separated list are split into separate skills.
- **Home:** greeting with verification status, readiness ring, the Career Action Plan ("Your next steps"), an "Eligible for you" carousel, "Closing soon" list and the latest announcement.
- **Drives:** search by title, company, location or skill; quick chips (Eligible, Open, Bookmarked); a filter and sort sheet (job type, role category, location, minimum package, best match / deadline / package); bookmarks stored locally.
- **Drive detail:** company, package and deadline countdown; explainable eligibility card; explainable match card; required skills; recruitment stages; an Apply button that is disabled with a reason when applying is not allowed.
- **Applications and journey:** status-filtered list and a vertical timeline showing completed, current, upcoming and failed stages with schedules, venues and officer notes.
- **Readiness:** overall score, five weighted components, six-week activity chart, suggestions, relevant upcoming drives, recent activity and an activity log dialog.
- **Profile:** name, academic record (locked after verification), skills with alias normalisation, preferred roles, certifications, resume link, LinkedIn, GitHub and summary.
- **Notifications:** application updates and announcements, with local Android notifications.

1.2 Placement Officer Portal

- **Dashboard:** open drives, total applications, pending verifications and selections this season, with quick actions and upcoming deadlines.
- **Verification queue:** pending and verified students with a review dialog to verify a complete record or request a correction (un-verify).
- **Companies and drives:** company management; a drive editor for criteria, branches, graduation years, required skills, preparation focus, deadline and an ordered stage list; save as draft, publish and close.
- **Applicants:** per-drive applicant list filtered by status, stage and name; applicant detail with profile snapshot, skills, links and timeline; Advance and Reject with optional notes.

- **Announcements and analytics:** announcement composer; publishing or reopening a drive automatically notifies every student; applications per drive, recruitment funnel, branch-wise selections, readiness of applicants versus selected students and preparation insights.

1.3 Explainable Eligibility Engine

EligibilityEngine evaluates CGPA, branch, active backlogs and graduation year in that order. Each criterion yields PASS, FAIL or UNKNOWN together with the student's value, the requirement and a sentence such as "Your CGPA is 6.99; this drive requires at least 7.00 (short by 0.01)." Any FAIL gives NOT ELIGIBLE; otherwise any UNKNOWN value or an unverified record gives NEEDS VERIFICATION; otherwise the student is ELIGIBLE. CGPA uses an epsilon comparison so that 7.00 against a 7.00 minimum passes. The same conditions are re-checked by the Firestore rule eligibleAndConsistent(), so the client explanation and the server decision agree.

1.4 Explainable Match Score

MatchScoreCalculator ranks eligible (or provisionally eligible) drives with the documented formula Match = 0.60·S + 0.25·P + 0.15·R, where S is the percentage of required skills the student has (after lower-casing and resolving aliases such as "js" → "javascript"), P is 100 for a preferred role, 50 when no preferences are set and 0 otherwise, and R is the mean readiness of the drive's preparation-focus components. The result includes the three component scores, matched and missing skills, plain-language reasons and a band (STRONG ≥ 75, GOOD ≥ 50, FAIR < 50). The interface states that the score ranks opportunities and does not predict selection.

1.5 Placement Readiness Dashboard

ReadinessAnalyzer uses the profile and activities from the last 30 days. Components and weights are: resume completeness 20% (resume link, three or more skills, a certification, a summary, GitHub or LinkedIn), aptitude 25% and coding 25% (average score scaled by min(count/4, 1)), mock interviews 15% (average rating scaled by min(count/2, 1)) and skill development 15% (minutes / 600, capped). It also returns six weekly activity counts, suggestions for the two weakest components and a drive-linked suggestion when eligible drives closing within seven days share a weak focus area.

1.6 Career Action Engine

CareerActionEngine turns the results already produced by the other engines into at most five ranked actions. It does not recompute eligibility or scores. Each action states why it appears and what to do next, and opens the relevant screen.

Action	Priority rule	Generated when	Opens
VERIFY	1000	Academic record unverified or incomplete	Profile

Action	Priority rule	Generated when	Opens
STAGE	900 − 10 × days to round	Active application has a scheduled round within 14 days	Application journey
APPLY (urgent)	800 − hours left / 6	Allowed to apply and the deadline is within 3 days	Drive detail
APPLY	500 + match − days left (max 30)	Allowed to apply (eligible, open, not applied)	Drive detail
SKILL	300 + 10 × drives needing it	Required skill missing for target drives	Readiness
READINESS	200 + (60 − component %)	Weakest focus component below 60%	Readiness
PROFILE	150	Resume completeness below 100%	Profile

Table 2: Career Action Engine ranking rules (at most 2 APPLY and 2 SKILL actions, 5 in total)

Apply actions are generated only when ApplicationWorkflow allows an application, so ineligible, closed, expired and already-applied drives are never recommended. Rejected and selected applications produce no stage reminders. The plan is recomputed whenever the profile, drives, applications, bookmarks or logged activities change.

1.7 Application Workflow

Applications use the deterministic document id {driveId}_{studentUid}. A second application by the same student is therefore an update, which the rules deny for students, so duplicates are rejected by the server. ApplicationWorkflow moves an application from SUBMITTED through IN_PROGRESS to SELECTED when the officer advances past the last round, or to REJECTED at any non-final stage. Final states are immutable in both the client and the rules. Each officer decision is one batched write that updates the application and creates a notification for the student.

1.8 Offline Behaviour

Firestore offline persistence is enabled, and Room additionally stores bookmarks, readiness activities (with a synced flag), cached drives and cached applications. When offline, the app shows an "Offline — showing saved data" banner and the Career Action Plan states that it is based on saved data. Activities logged offline are uploaded once by SyncWorker when the network returns; their UUID document ids make the upload idempotent.

1.9 Announcements, Notifications and Analytics

Officers publish announcements to all students, and publishing or reopening a drive automatically sends a "New drive" announcement linked to that drive; officer decisions create targeted notifications. While the app is open, a Firestore listener posts local notifications; in the background, SyncWorker checks every 15 minutes whenever a network connection is available. The officer analytics screen shows applications per drive, a per-drive recruitment funnel, branch-wise selections, readiness of applicants versus selected students, and

preparation insights: average readiness per component and the most common missing skills among verified students eligible for open drives. Insights are aggregate counts only and are withheld when fewer than three students are covered.

2. Application Screens

Table 3 lists the screens, their purpose and how they are reached. Screenshots of each screen are given in Chapter 6, and Figure 5 in Chapter 3 shows the navigation structure.

Screen	Role	Purpose	Navigation
Onboarding	All	Three introductory slides	First launch → Skip / Next → Sign in
Sign in / Create account	All	Authentication, registration as student or officer, password reset	After onboarding; after logout
Home	Student	Next steps, readiness, eligible and closing-soon drives, announcement	Bottom navigation: Home
Drives	Student	Search, quick filters, filter and sort sheet, bookmarks	Bottom navigation: Drives
Drive details	Student	Eligibility card, match card, company, stages, Apply	Tap a drive anywhere
Applications	Student	Applications by status	Bottom navigation: Applications
Application journey	Student	Recruitment timeline and profile snapshot	Tap an application or a STAGE action
Readiness	Student	Score, components, trend, suggestions, activity log	Bottom navigation: Readiness
Profile	Student	Academic record and career profile	Bottom navigation: Profile
Notifications	Student	Updates and announcements	Toolbar bell; tapping a notification
Dashboard	Officer	Metrics, quick actions, pending verifications, deadlines	Bottom navigation: Dashboard
Verification queue	Officer	Verify or un-verify academic records	Bottom navigation: Verify
Placement drives / Companies	Officer	Drive list by status; company management	Bottom navigation: Drives; toolbar icon
Applicants / Applicant	Officer	Applicant list and detail with Advance / Reject	Tap a drive; tap an applicant
Drive editor	Officer	Create or edit, publish or close a drive	New drive; Edit drive
Announcements	Officer	List and compose announcements	Bottom navigation: Announce
Analytics	Officer	Charts, funnel, readiness comparison, preparation insights	Bottom navigation: Analytics

Table 3: Application screens and navigation

3. System Design and Blueprints

The diagrams in this chapter were drawn from the actual package structure, classes, Firestore collections and Security Rules in the repository. They are shown on landscape pages so that their labels remain legible when printed.

3.1 System Architecture

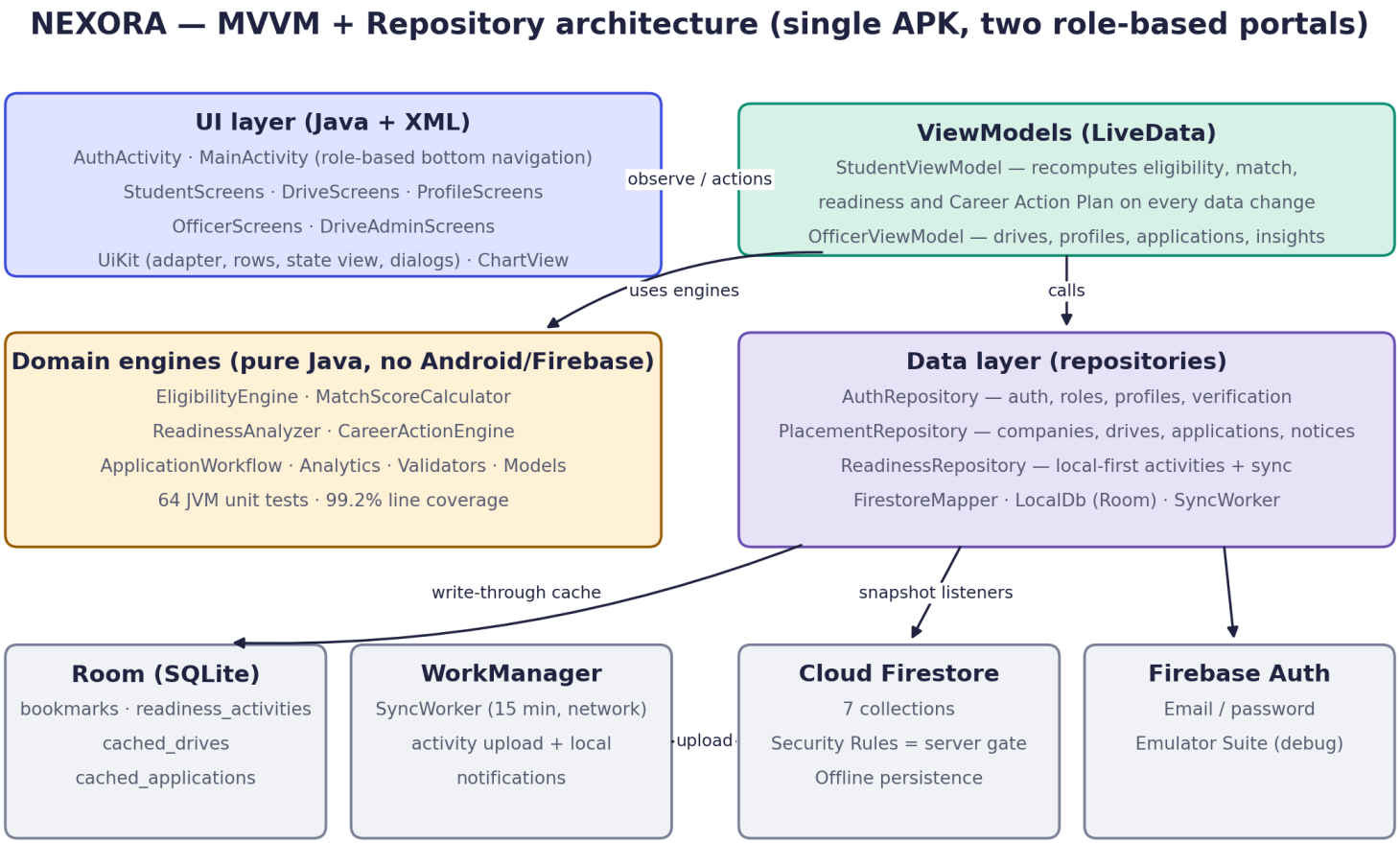


Figure 1: *NEXORA system architecture (MVVM + Repository)*

3.2 Module Structure

The application consists of 28 Java source files (about 6,500 lines) in three packages. Domain classes import no Android or Firebase classes; a unit test scans the domain package to enforce this.

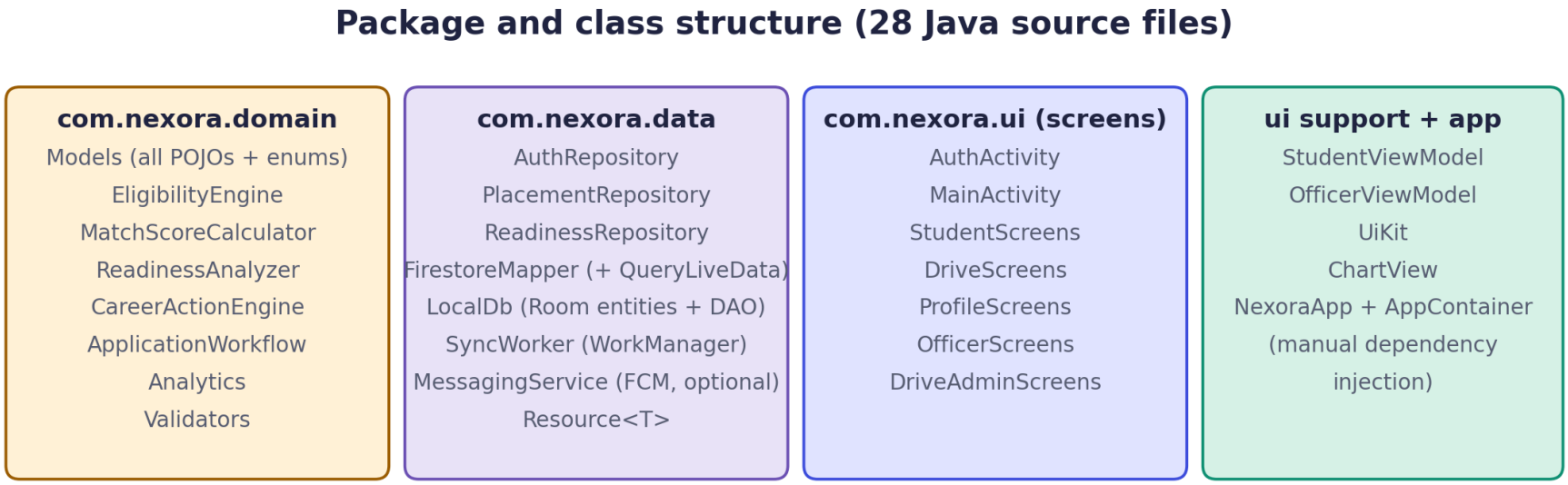


Figure 2: *Package and class structure*

3.3 Use Cases

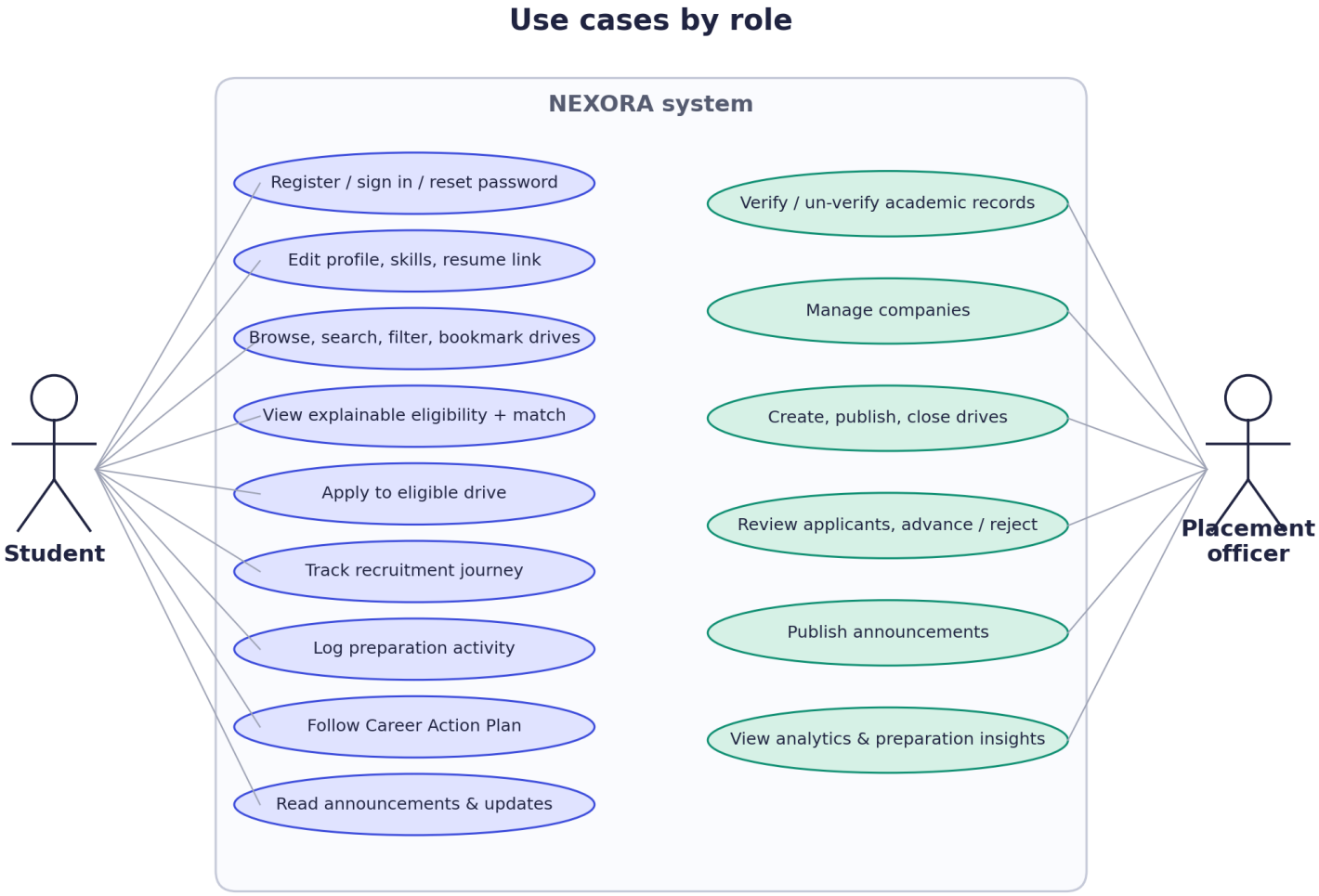


Figure 3: *Use-case diagram for students and placement officers*

3.4 Integrated Workflow

Each stage consumes the results of the previous one: verified profile data drives eligibility, eligibility gates the match score, both feed the Career Action Engine, logged preparation updates readiness, applications move through the recruitment journey, and the officer analytics aggregate the outcome. Any change triggers recomputation of the student's plan.

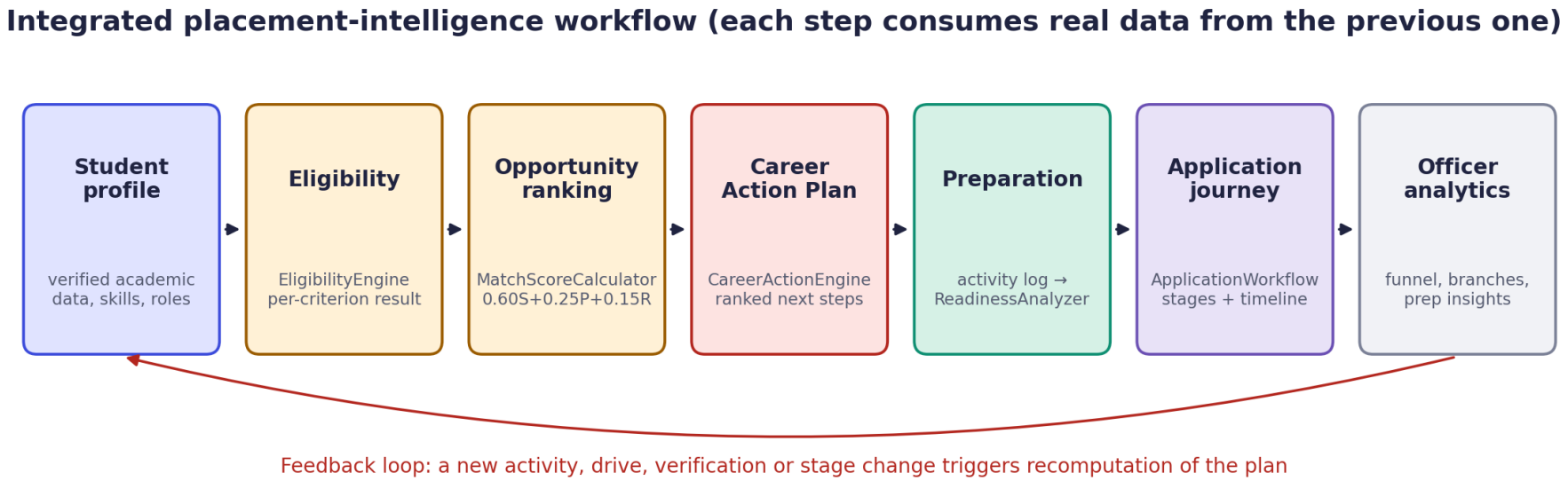


Figure 4: *Integrated placement-intelligence workflow*

3.5 Screen Navigation

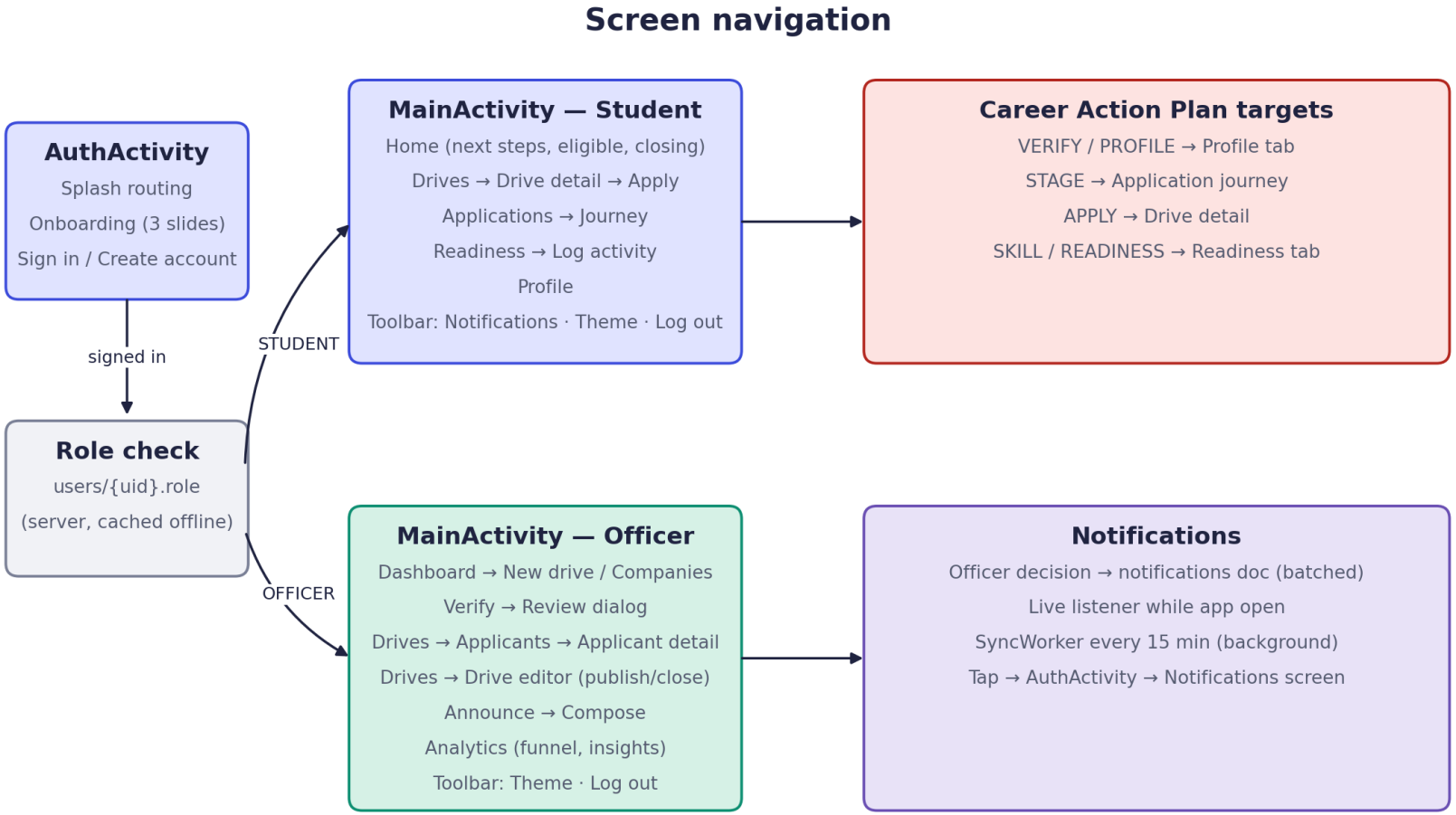


Figure 5: *Screen navigation for both roles and Career Action Plan targets*

3.6 Application Lifecycle

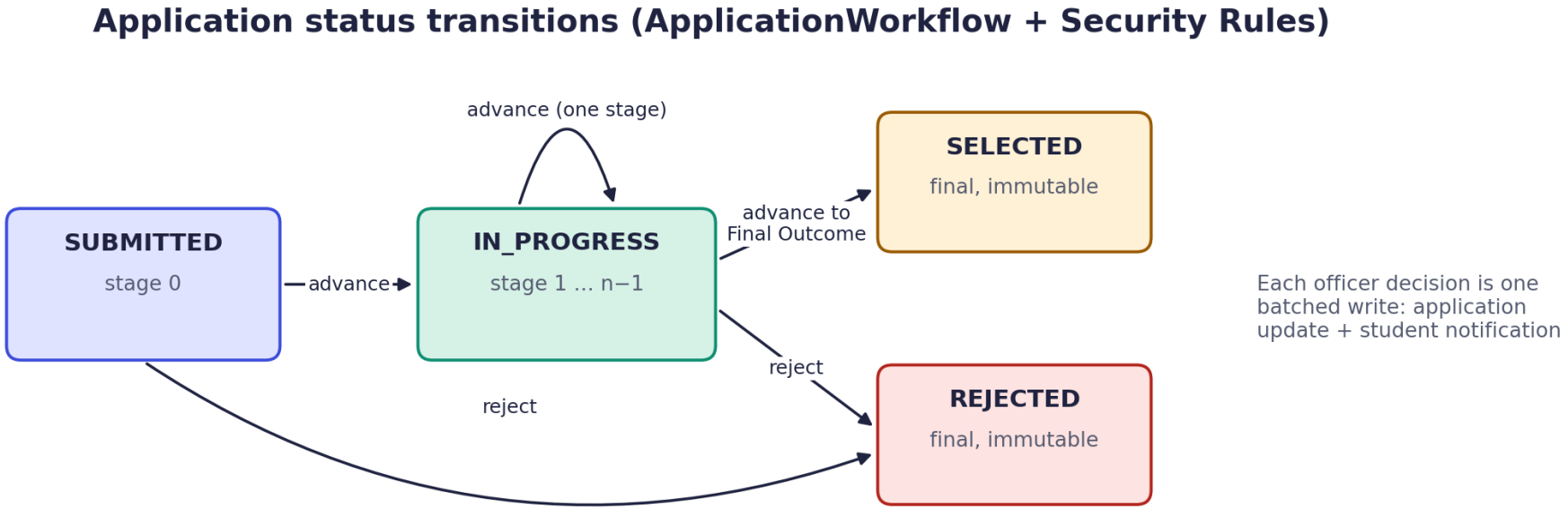


Figure 6: *Application status transitions*

3.7 Data Model and Security

Cloud Firestore stores seven collections (users, studentProfiles, companies, placementDrives, applications, readinessProgress/{uid}/activities and notifications). Room stores bookmarks, readiness activities and cached copies of drives and applications. Figure 7 summarises the collections and the main rules protecting them.

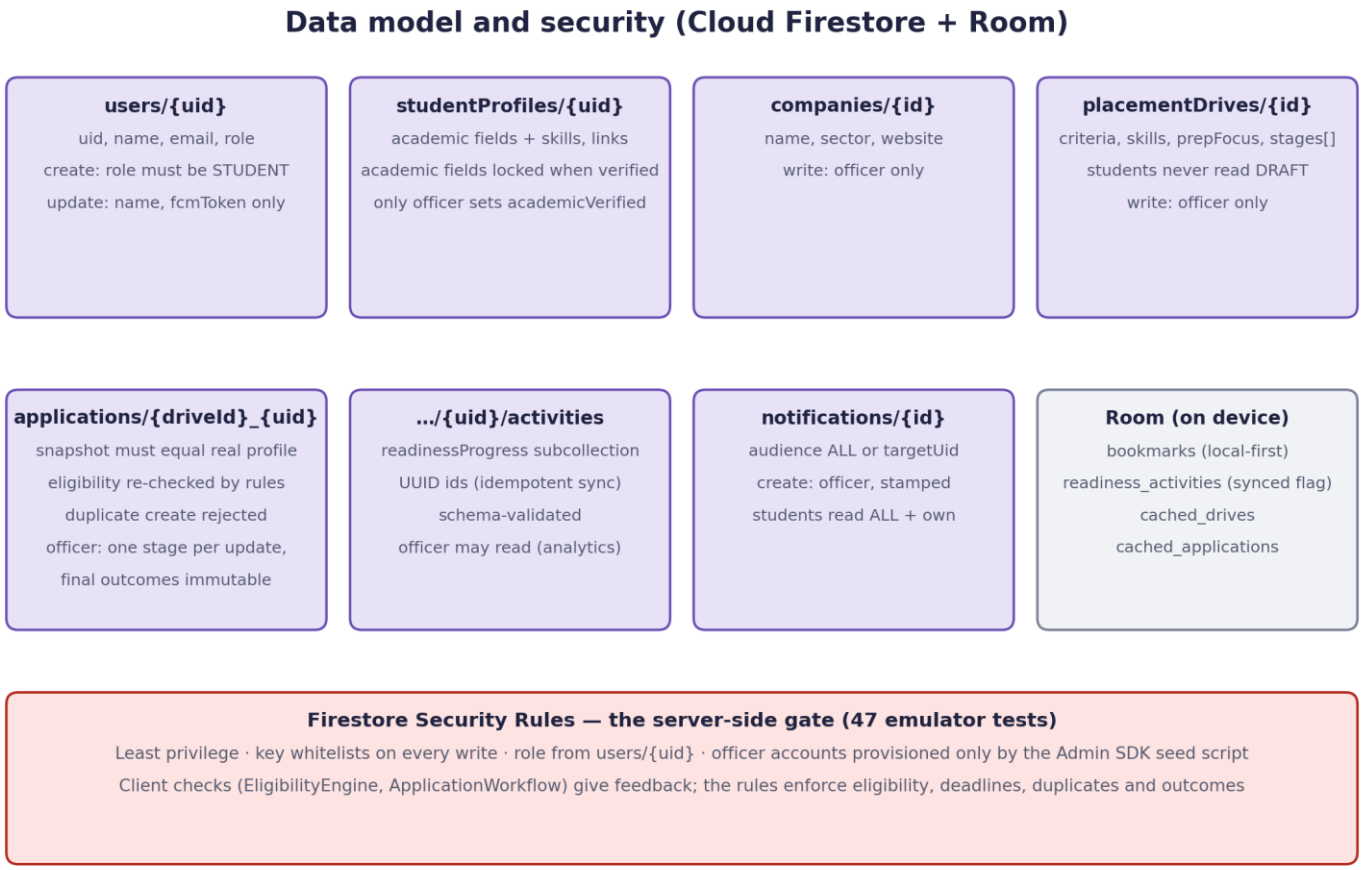


Figure 7: *Data model and Firestore Security Rules*

4. Implementation and Code

4.1 Key Java Classes

Class (package)	Responsibility
Models (domain)	All data classes and enums as nested static classes (StudentProfile, Drive, Stage, Application, Activity, Notice …)
EligibilityEngine (domain)	Per-criterion eligibility with explanations; mirrors the eligible() rule
MatchScoreCalculator (domain)	Weighted match score, skill normalisation and aliases, reasons and bands
ReadinessAnalyzer (domain)	Readiness components, overall score, weekly trend and suggestions
CareerActionEngine (domain)	Ranked, explained next steps built from the other engines' results
ApplicationWorkflow, Analytics, Validators (domain)	Apply decision and stage transitions; officer metrics and insights; input validation
AuthRepository, PlacementRepository, ReadinessRepository (data)	Firebase and Room access exposed as LiveData; batched officer decisions; offline cache
FirestoreMapper, LocalDb, SyncWorker (data)	Document ↔ model mapping and lifecycle-aware listeners; Room database; background sync and local notifications
StudentViewModel, OfficerViewModel (ui)	Combine live data and recompute all derived results
AuthActivity, MainActivity, *Screens, UiKit, ChartView (ui)	Screens, role-based navigation, reusable UI components and a custom chart view

Table 4: Key Java classes

4.2 XML Layouts and Resources

The interface uses 22 XML layouts. Reusable layouts reduce duplication: fragment_list.xml serves every list screen (search, chips, list, state view and floating action button); item_row.xml is the common card row; item_criterion.xml displays eligibility criteria, match reasons, suggestions and Career Action Plan entries; item_timeline.xml draws the recruitment timeline; and view_state.xml provides loading, empty and error states. Colours are defined as light and dark tokens (values/colors.xml and values-night/colors.xml) applied through a Material 3 theme.

4.3 Firebase Integration

NexoraApp initialises Firebase from the options in app/google-services.json when present; the build then targets the live project (nexora-a014b). Without the file, debug builds connect to the local Firebase Emulator Suite under the offline demo-nexora project. The file is excluded from version control, so no credentials are stored in the repository. Repositories expose Firestore queries through a LiveData subclass that attaches the snapshot listener in onActive() and removes it in onInactive(), preventing listener leaks. The rules and composite indexes were deployed to the live project with the Firebase CLI.

4.4 Code Excerpts

The following excerpts are taken verbatim from the repository.

Listing 1: CGPA criterion with explanation — *domain/EligibilityEngine.java*

```
private CriterionResult cgpa(StudentProfile p, Drive d) {
    String req = "≥ " + fmt2(d.minCgpa);
    if (p.cgpa == null) return unknown(Criterion.CGPA, req, "CGPA is missing from your profile.");
    String mine = fmt2(p.cgpa);
    if (p.cgpa + EPS >= d.minCgpa) {
        return new CriterionResult(Criterion.CGPA, Outcome.PASS, mine, req,
                "Your CGPA is " + mine + "; this drive requires at least " + fmt2(d.minCgpa) + ".");
    }
    return new CriterionResult(Criterion.CGPA, Outcome.FAIL, mine, req,
            "Your CGPA is " + mine + "; this drive requires at least " + fmt2(d.minCgpa)
                    + " (short by " + fmt2(d.minCgpa - p.cgpa) + ").");
}
```

The method returns PASS or FAIL with the student value, the requirement and an explanation; the epsilon makes the boundary inclusive.

Listing 2: Weighted match score — *domain/MatchScoreCalculator.java*

```
        }
    }
    r = sum / focus.size();
}


res.s = (int) Math.round(s);
```

Listing 3: Career Action Engine ranking — *domain/CareerActionEngine.java*

```
public List<Action> plan(StudentProfile p, List<? extends Opportunity> opps,
                         ReadinessAnalyzer.Report readiness, long now) {
    List<Action> out = new ArrayList<>();
    verification(p, out);
    stages(opps, now, out);
    applications(opps, now, out);
    List<Opportunity> targets = targets(opps, now);
    skills(targets, out);
    readiness(readiness, targets, out);


    out.sort(Comparator.comparingInt((Action a) -> -a.priority).thenComparing(a -> a.title));
    List<Action> ranked = new ArrayList<>();
    int apply = 0, skill = 0;
    for (Action a : out) {
        if (ranked.size() == MAX_ACTIONS) break;
        if (a.kind == Kind.APPLY && apply++ >= 2) continue;
        if (a.kind == Kind.SKILL && skill++ >= 2) continue;
        ranked.add(a);
    }
    return ranked;
}
```

Each rule adds candidate actions; the list is sorted by priority, capped at five actions and limited to two apply and two skill actions.

Listing 4: Only allowed applications become recommendations — *domain/CareerActionEngine.java*

```
private static void applications(List<? extends Opportunity> opps, long now, List<Action> out) {
    for (Opportunity o : opps) {
        // Only drives the workflow already allows: eligible, open, before deadline, not applied.
        if (o.decision == null || !o.decision.allowed || o.match == null) continue;
        long left = o.drive.deadline.getTime() - now;
        int days = (int) (left / DAY);
        boolean urgent = left <= URGENT_DAYS * DAY;
        Action act = new Action(Kind.APPLY,
                urgent ? 800 - (int) (left / (60 * 60 * 1000L)) / 6 : 500 + o.match.score -
Math.min(days, 30),
                (urgent ? "Apply soon: " : "Apply: ") + o.drive.title + " · " + o.drive.companyName,
                "You're eligible with a " + o.match.score + "% match, and applications close " +
when(days) + ".",
                "Open the drive to review details and apply.");
        act.driveId = o.drive.id;
        out.add(act);
    }
```

Listing 5: Deterministic application id and server rejection — *data/PlacementRepository.java*

```
/**
 * Creates the application with the deterministic id {driveId}_{uid}. A second attempt is an
 * update, which rules deny for students, so duplicates are rejected by the server.
 */
public void apply(Application a, Resource.Callback<Void> cb) {
    Map<String, Object> m = FirestoreMapper.applicationMap(a);
    m.put("appliedAt", FieldValue.serverTimestamp());
    m.put("updatedAt", FieldValue.serverTimestamp());
    db.collection(FirestoreMapper.APPLICATIONS).document(a.id).set(m)
            .addOnSuccessListener(v -> cb.onResult(Resource.success(null)))
            .addOnFailureListener(e -> {
                boolean denied = e instanceof FirebaseFirestoreException
                        && ((FirebaseFirestoreException) e).getCode() ==
FirebaseFirestoreException.Code.PERMISSION_DENIED;
                cb.onResult(Resource.error(denied
                        ? "The server rejected this application. You may have already applied, the
drive may be closed or past its deadline, or your verified record does not meet the criteria."
                        : FirestoreMapper.errorMessage(e), null));
            });
}
```

Listing 6: Career Action Plan card on the home screen — *res/layout/fragment_home.xml*

```
<!-- Career Action Plan: ranked next steps from CareerActionEngine. -->
<TextView
    style="@style/Nexora.Section"
    android:text="@string/next_steps" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <LinearLayout
        style="@style/Nexora.CardBody"
        android:paddingVertical="8dp">

        <TextView
            android:id="@+id/actions_note"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:paddingVertical="4dp"
            android:textAppearance="@style/TextAppearance.Nexora.Caption"
            android:visibility="gone" />

        <LinearLayout
            android:id="@+id/actions_list"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

4.5 Firestore Security Rules

The rules are the authoritative gate because the client is untrusted. Registration can create only the STUDENT or OFFICER role, and the role can never be changed afterwards from the client (open officer sign-up is a deliberate project decision; a production deployment would add officer approval); academic fields cannot change after verification; only officers can verify records (stamped with their own uid and the server time), write drives and companies, record outcomes and publish notifications; students cannot read draft drives or other students' data. Application creation re-checks eligibility, the deadline and the drive status, and requires the denormalised snapshot, names and titles to equal the real profile and drive.

Listing 7: Server-side eligibility and integrity check for applications — *firebase/firestore.rules*

```
// plus integrity of every denormalised field against the real drive and profile.
function eligibleAndConsistent(driveId, uid) {
  let d = get(/databases/$(db)/documents/placementDrives/$(driveId)).data;
  let p = get(/databases/$(db)/documents/studentProfiles/$(uid)).data;
  let a = request.resource.data;
  return d.status == 'OPEN'
    && request.time < d.deadline
    && p.academicVerified == true
    && p.cgpa >= d.criteria.minCgpa
    && p.branch in d.criteria.eligibleBranches
    && p.activeBacklogs <= d.criteria.maxActiveBacklogs
    && p.graduationYear in d.criteria.graduationYears
    && a.driveTitle == d.title
    && a.companyName == d.companyName
    && a.studentName == p.name
    && a.studentRoll == p.rollNumber
    && a.profileSnapshot == {'cgpa': p.cgpa, 'branch': p.branch,
```

5. Testing and Results

5.1 Automated Testing

All automated suites below were executed in the project environment on 26 October 2026 and passed. The instrumented suite was last run before the final sign-up screen change; the unit and rules suites were re-run after it.

Suite	Tool / environment	Tests	Result
JVM unit tests (domain)	JUnit 4, ./gradlew testDebugUnitTest	65	65 passed, 0 failed
Domain code coverage	JaCoCo (createDebugUnitTestCoverageReport)	—	99.2% of lines (645 of 650)
Instrumented tests	AndroidX Test / Espresso on Pixel 6a emulator (Android 15, API 35)	6	6 passed (OK)
Firestore Security Rules tests	@firebase/rules-unit-testing + Firebase Emulator Suite (npm test)	48	48 passed, 0 failed
Debug build	./gradlew assembleDebug	—	BUILD SUCCESSFUL

Table 5: Automated test results

Test file	Tests	Coverage
EligibilityMatchTest.java	25	Eligibility boundaries (7.00 vs 7.00 passes, 6.99 fails, backlog limits, missing values, unverified), match score weights, aliases, bands, comma-separated skill input
ReadinessWorkflowTest.java	25	Readiness formulas, 30-day window, weekly buckets, workflow transitions, and Career Action Engine cases: ineligible, applied, closed and expired drives excluded; urgent ranking; skill gaps; readiness updates; verification; stage reminders
ValidatorsAnalyticsTest.java	15	Input validators, dashboard and funnel counts, preparation insights (aggregate only, three-student threshold, empty data), domain purity check
LocalDbTest.java (instrumented)	4	Room DAO: bookmarks, idempotent activity sync, drive cache round-trip, per-user application cache
AuthFlowTest.java (instrumented)	1	Onboarding and registration validation messages (Espresso)
UiKitTest.java (instrumented)	1	Field errors clear as soon as input becomes valid
rules.test.js (Firestore rules)	48	Officer sign-up allowed but other roles rejected, officer cannot create a student profile, role changes blocked, verification locking, private data, draft drives, valid and invalid applications, duplicates, deadlines, officer-only operations, notifications

Table 6: Automated test inventory

5.2 Manual Testing

Manual end-to-end testing was performed on the Pixel 6a emulator against the seeded Firebase Emulator Suite. These checks were done by hand (driven through adb) and are distinct from the automated suites above.

- Login with a wrong password shows "Incorrect email or password."; role-based routing opens the correct portal.
- A student with CGPA 6.99 sees "short by 0.01" for a 7.00 drive and cannot apply; verified academic fields are locked.
- An officer verified an unverified student, whose drives then became eligible; an officer-created drive appeared for students with a match score.
- Applications were accepted by the rules; the Apply button then changed to "Applied"; an officer advance updated the timeline and produced an Android notification.
- Career Action Plan ranking and navigation were checked for a verified and an unverified student; offline mode showed the stale-data notes and recovered online.
- An activity logged offline was uploaded exactly once after reconnecting and remained single after a forced re-sync.
- Officer analytics and preparation insights matched values calculated by hand from the seed data; no student names or roll numbers appear in the insights.

Live deployment (real Firebase project, two devices)

The officer portal ran on the Android emulator and the student portal on a physical Nothing phone (Android 15), both connected to the live project. All seven checks passed:

- Officer published a drive; the phone received "New drive: Software Development Intern · TechNova Solutions" and the drive appeared in the student's plan.
- Officer verified the student; the phone updated live to Eligible with a 65% match (0.60 × 67 + 0.25 × 100 + 0.15 × 0) and Apply enabled.
- Student applied from the phone; the live rules accepted it and the officer dashboard updated immediately.
- Officer advanced the applicant with a note; the phone received the stage notification and showed the application in progress.
- Officer announcement reached the phone through the background sync while the app was closed.
- Officer analytics showed the real data, with preparation insights withheld below three students; neither device logged permission, index or crash errors.

5.3 Limitations of Testing

The full register-to-apply and officer create-to-publish journeys were tested manually, not as automated Espresso tests. Background notifications on a physical phone can be delayed by Android battery optimisation when the app is closed; they are immediate while the app is open. No usability study with students was conducted.

6. Output Screenshots

All screenshots are genuine captures of the running application on the Pixel 6a emulator (Android 15) with the seeded demo data. Seeded accounts: Aisha (verified, eligible), Rahul (verified, CGPA 6.99), Kiran (unverified) and the placement officer.

6.1 Student Portal

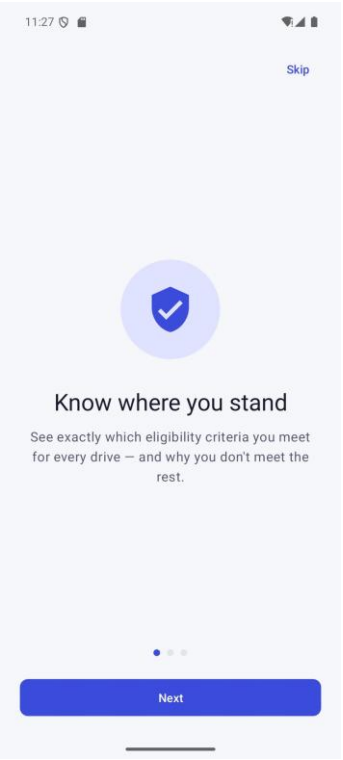


Figure 8: *Onboarding*

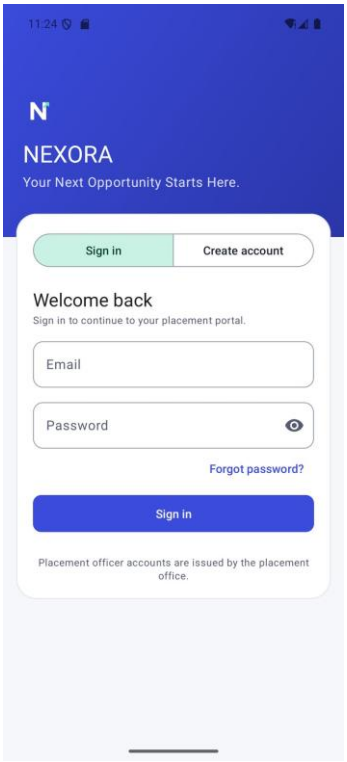


Figure 9: *Sign in*

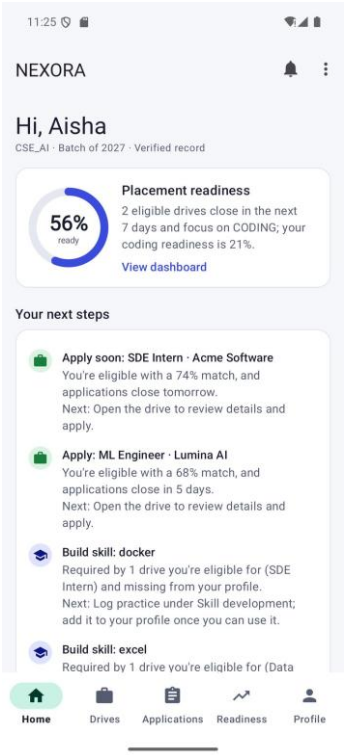


Figure 10: *Student home*

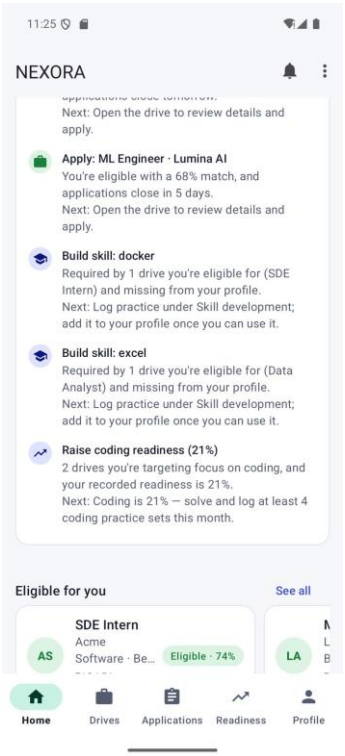


Figure 11: *Career Action Plan*

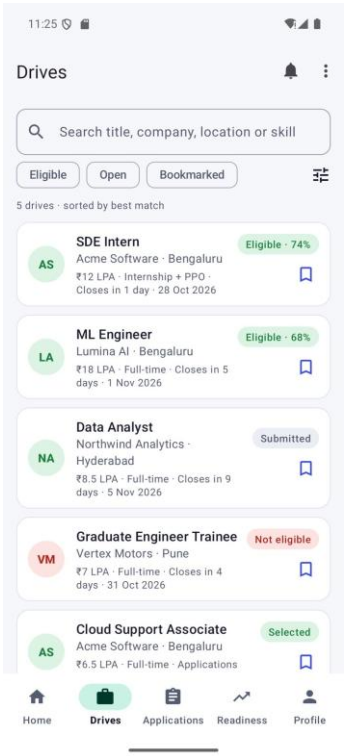


Figure 12: *Drives with eligibility badges*

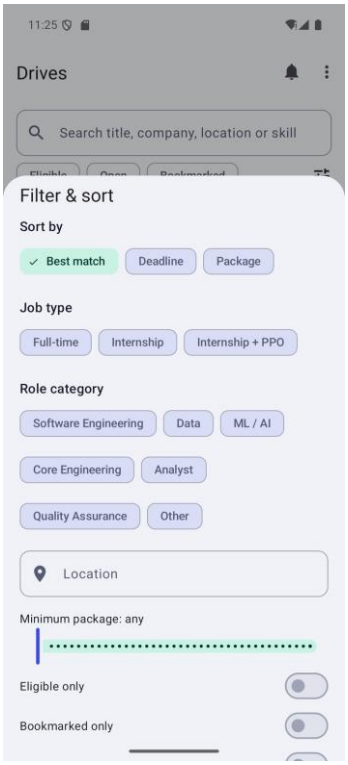


Figure 13: *Filter and sort sheet*

Figures 8–13: the home screen shows readiness and the ranked next steps (urgent application first, then skill gaps and readiness); the drives list shows per-drive eligibility and match badges.

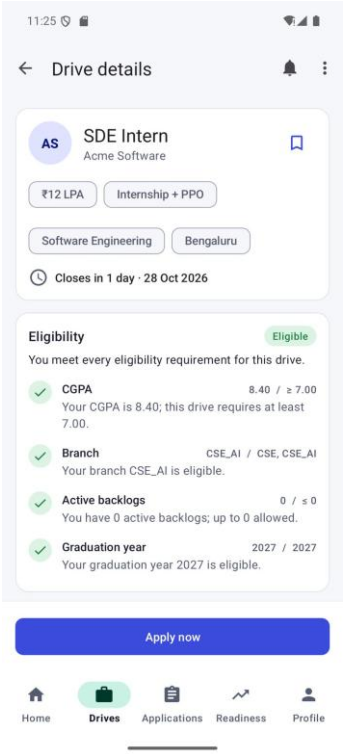


Figure 14: *Explainable eligibility*

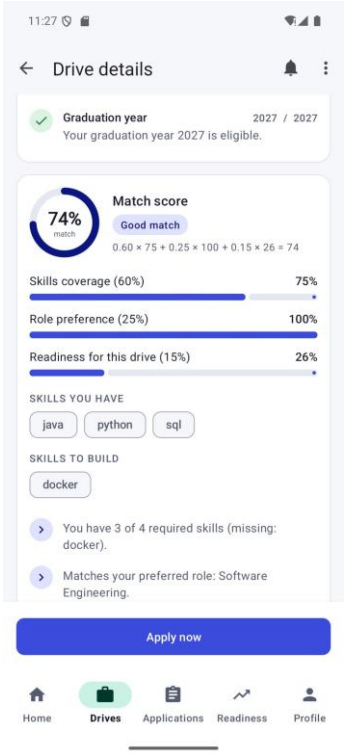


Figure 15: *Explainable match score*

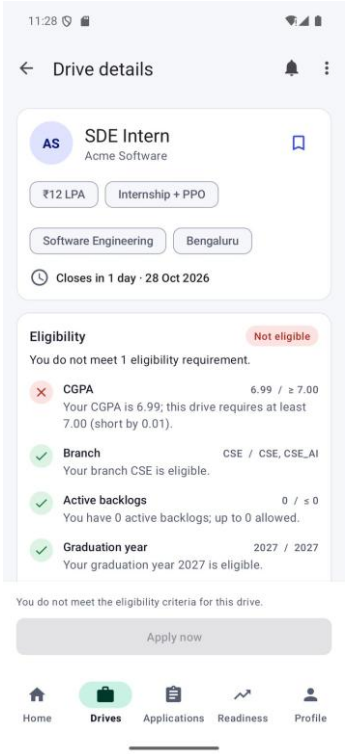


Figure 16: *Not eligible: CGPA 6.99*

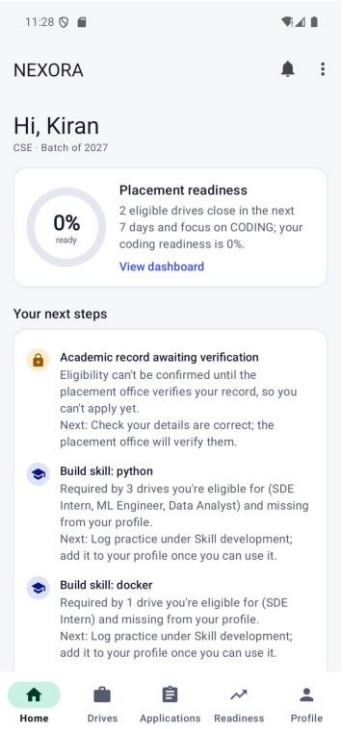


Figure 17: *Unverified student plan*

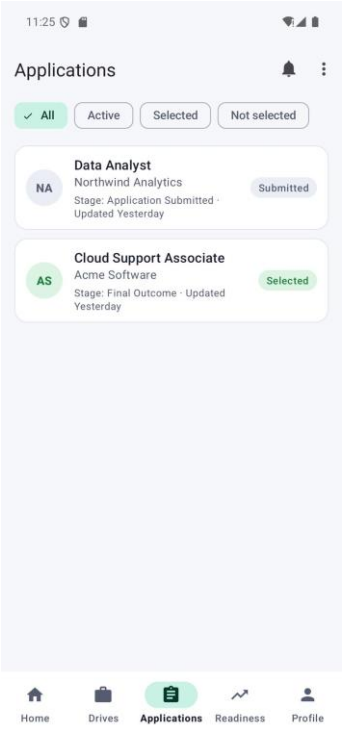


Figure 18: *Applications*

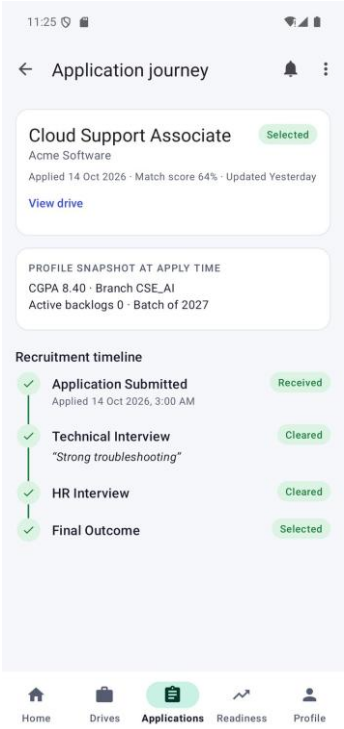


Figure 19: *Recruitment journey*

Figures 14–19: the eligibility card explains each criterion; the match card shows 0.60 × 75 + 0.25 × 100 + 0.15 × 26 = 74; Rahul's CGPA gap of 0.01 blocks Apply; an unverified student's plan starts with verification; the journey shows each completed stage with officer notes.

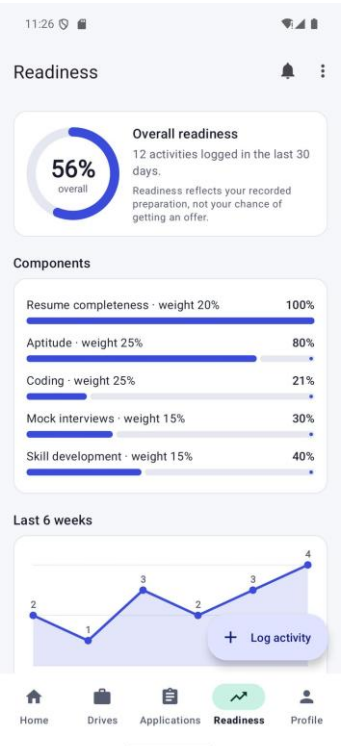


Figure 20: *Readiness dashboard*

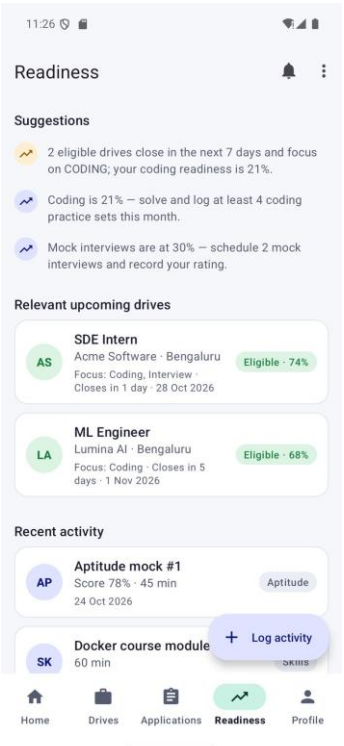


Figure 21: *Suggestions and relevant drives*

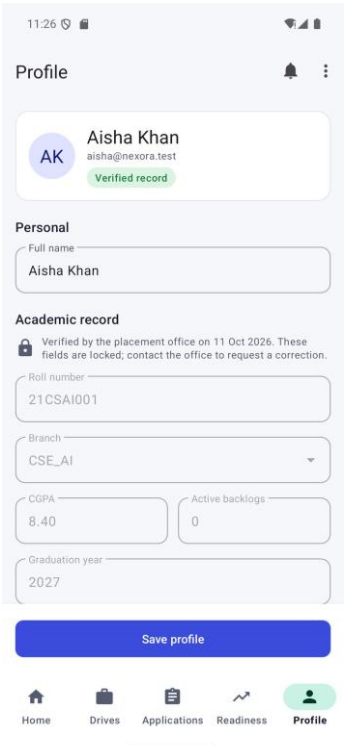


Figure 22: *Profile with locked academic record*

Figures 20–22: readiness components with their weights, a six-week trend, drive-linked suggestions, and the verified record locked against editing.

6.2 Placement Officer Portal

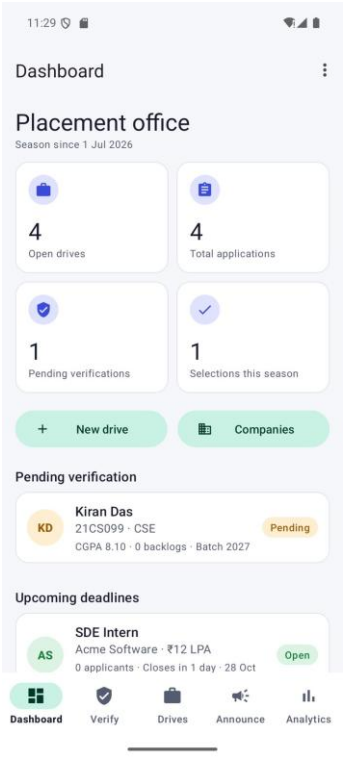


Figure 23: *Officer dashboard*

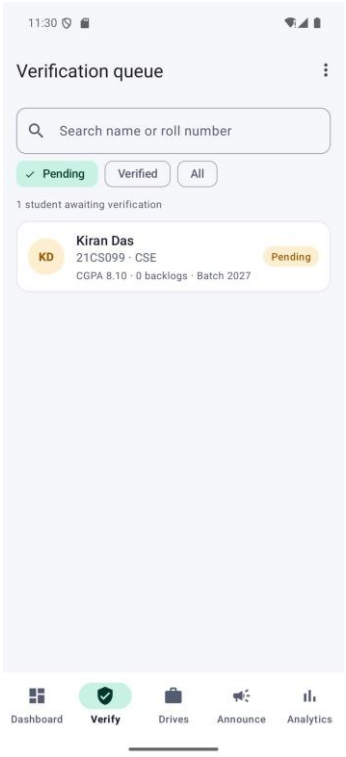


Figure 24: *Verification queue*

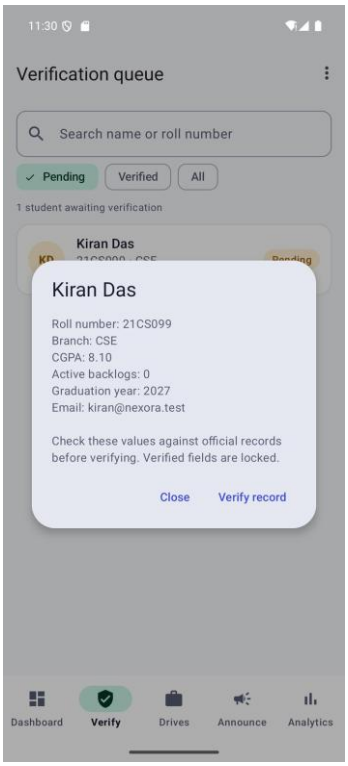


Figure 25: *Record review dialog*

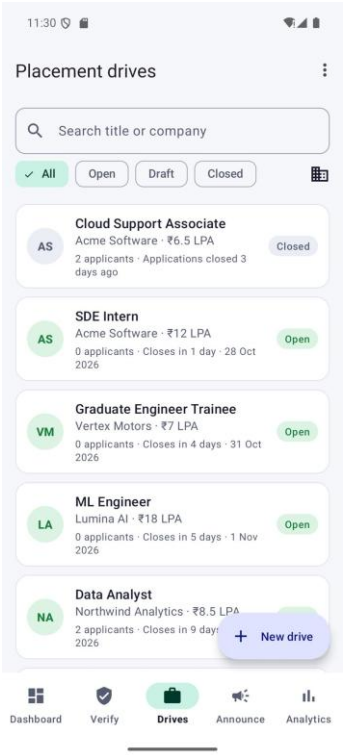


Figure 26: *Placement drives*

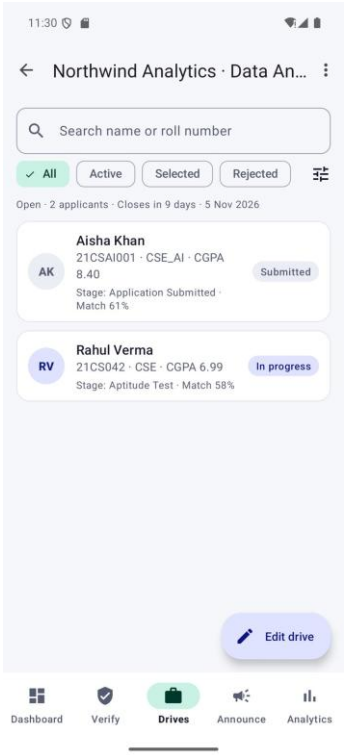


Figure 27: *Applicants for a drive*

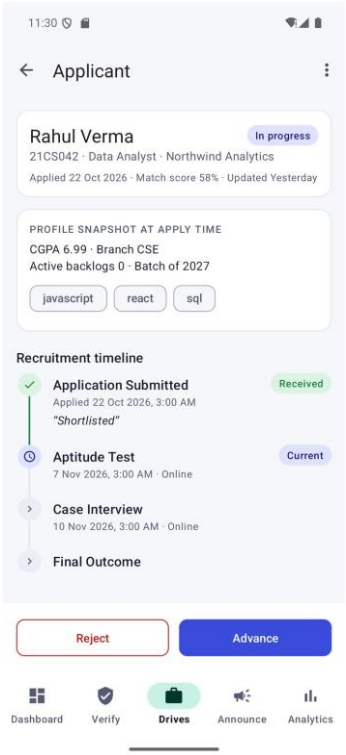


Figure 28: *Applicant detail with actions*

Figures 23–28: the officer reviews a complete record before verifying it, manages drives by status and advances or rejects applicants stage by stage.

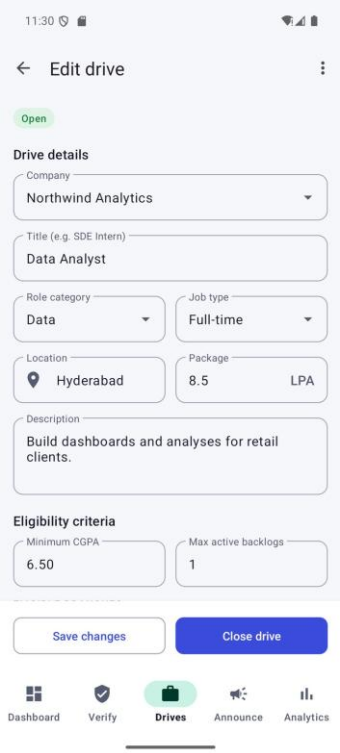


Figure 29: *Drive editor*

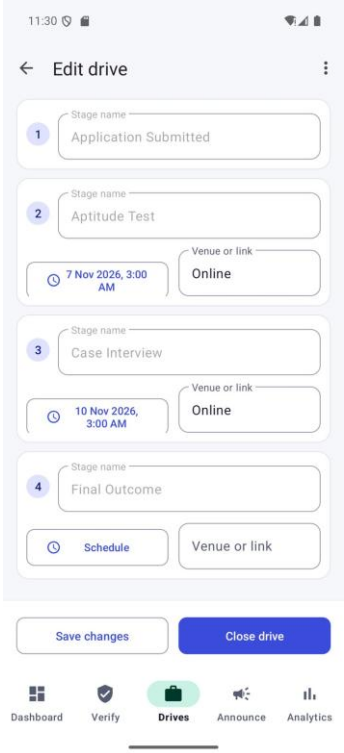


Figure 30: *Stage schedules*

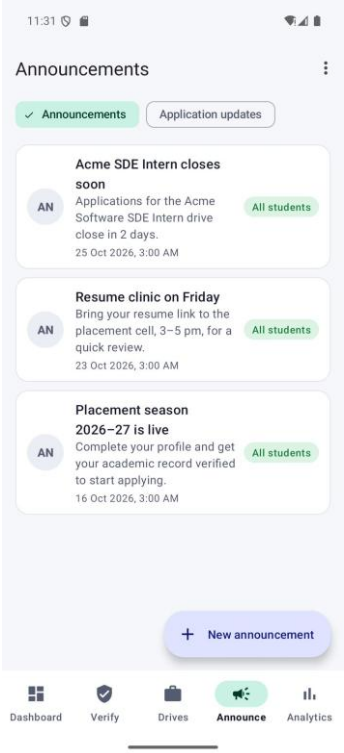


Figure 31: *Announcements*

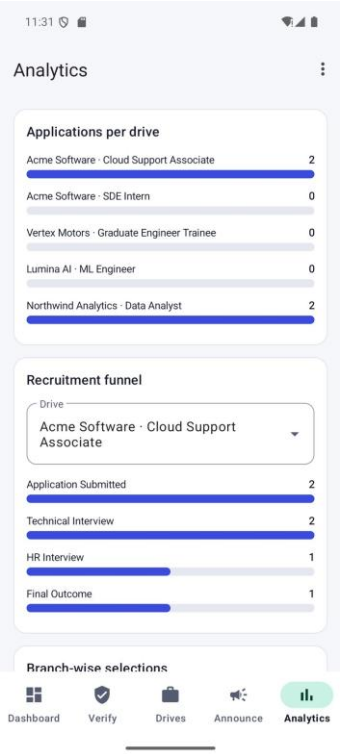


Figure 32: *Analytics: per drive and funnel*



Figure 33: *Preparation insights*

Figures 29–33: drive criteria and schedules, published announcements, and analytics including the aggregate-only preparation insights across four students.

6.3 Live Deployment

These screenshots come from the live Firebase project: the student portal on a physical phone (dark theme) and the officer portal on the emulator.

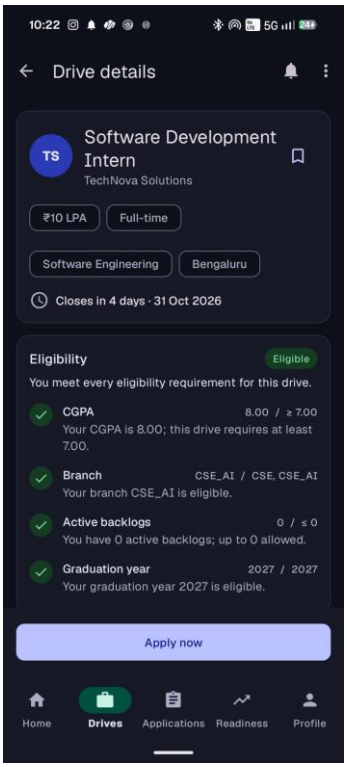


Figure 34: *Phone: live drive, Eligible*

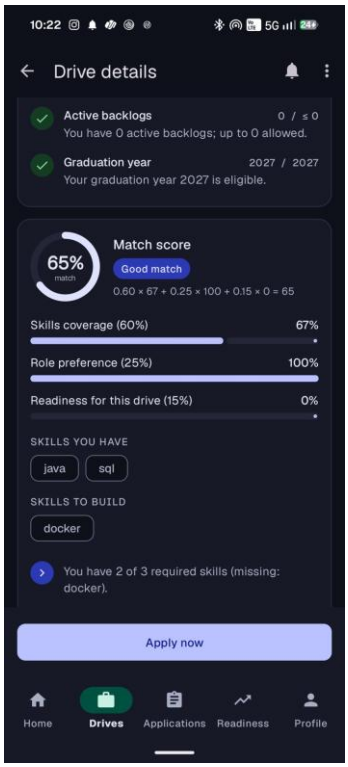


Figure 35: *Phone: live match score*

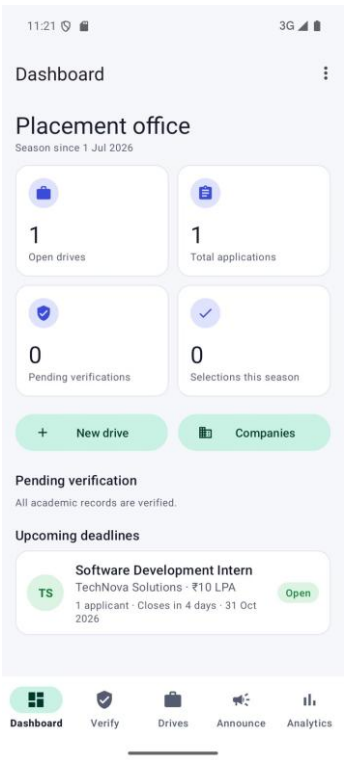


Figure 36: *Emulator: live officer dashboard*

Figures 34–36: after the officer verified the student, the phone showed the drive as Eligible with a 65% match, and the officer dashboard counted the live drive and application.

7. Conclusion and Future Scope

7.1 Conclusion

NEXORA delivers a complete campus placement workflow on Android using Java and XML. Its distinctive contribution is explainability combined with server-side trust: students see precisely why they are eligible or not, why a drive is ranked as it is and what to do next, while the Firestore Security Rules ensure that roles, verification, eligibility, deadlines, duplicate prevention and recruitment outcomes cannot be bypassed by a modified client. The Career Action Engine connects the eligibility, ranking, readiness and recruitment features into one guided workflow, and the officer portal turns the same data into actionable, privacy-preserving analytics. The domain logic is fully separated from Android and Firebase and is verified by automated tests, and the whole system runs on the free Firebase Spark plan with offline support. The application is deployed on a live Firebase project and was demonstrated across two devices.

7.2 Future Scope

- An officer approval step instead of open officer sign-up.
- Officer-configurable match-score weights and readiness targets stored in Firestore.
- Instant push notifications through a trusted server (for example Cloud Functions on the paid plan).
- Resume upload and parsing, and verified skill assessments instead of self-logged practice.
- Automated end-to-end Espresso journeys and a usability study with students.
- Calendar integration for recruitment rounds and export of applicant lists.
- A web dashboard for placement officers and multi-campus scaling of the analytics.

8. References

[1] Android Developers, "Guide to app architecture." https://developer.android.com/topic/architecture (accessed 26 Oct 2026).

[2] Android Developers, "ViewModel overview." https://developer.android.com/topic/libraries/architecture/viewmodel (accessed 26 Oct 2026).

[3] Android Developers, "LiveData overview." https://developer.android.com/topic/libraries/architecture/livedata (accessed 26 Oct 2026).

[4] Android Developers, "Save data in a local database using Room." https://developer.android.com/training/data-storage/room (accessed 26 Oct 2026).

[5] Android Developers, "WorkManager." https://developer.android.com/topic/libraries/architecture/workmanager (accessed 26 Oct 2026).

[6] Android Developers, "Espresso." https://developer.android.com/training/testing/espresso (accessed 26 Oct 2026).

[7] Android Developers, "Android Studio." https://developer.android.com/studio (accessed 26 Oct 2026).

[8] Google, "Material Design 3." https://m3.material.io/ (accessed 26 Oct 2026).

[9] Material Components for Android (GitHub repository). https://github.com/material-components/material-components-android (accessed 26 Oct 2026).

[10] Firebase, "Get started with Firebase Authentication on Android." https://firebase.google.com/docs/auth/android/start (accessed 26 Oct 2026).

[11] Firebase, "Cloud Firestore." https://firebase.google.com/docs/firestore (accessed 26 Oct 2026).

[12] Firebase, "Get started with Cloud Firestore Security Rules." https://firebase.google.com/docs/firestore/security/get-started (accessed 26 Oct 2026).

[13] Firebase, "Firebase Local Emulator Suite." https://firebase.google.com/docs/emulator-suite (accessed 26 Oct 2026).

[14] Firebase, "Build unit tests (Security Rules)." https://firebase.google.com/docs/rules/unit-tests (accessed 26 Oct 2026).

[15] Firebase, "Add the Firebase Admin SDK to your server." https://firebase.google.com/docs/admin/setup (accessed 26 Oct 2026).

[16] Firebase, "Firebase pricing (Spark plan)." https://firebase.google.com/pricing (accessed 26 Oct 2026).

[17] JUnit 4. https://junit.org/junit4/ (accessed 26 Oct 2026).

[18] JaCoCo Java Code Coverage Library. https://www.jacoco.org/jacoco/ (accessed 26 Oct 2026).

[19] Gradle, "Version catalogs." https://docs.gradle.org/current/userguide/version_catalogs.html (accessed 26 Oct 2026).

[20] Node.js, "Test runner." https://nodejs.org/api/test.html (accessed 26 Oct 2026).

[21] Amrita Vishwa Vidyapeetham (source of the official logo). https://www.amrita.edu/ (accessed 26 Oct 2026).

9. Appendix

A. Build, Run and Test Commands

Commands (run from the project root) — *development workflow*

```
cd firebase && npm install
cd firebase && npm run emulators     # Auth + Firestore emulators (demo-nexora)
cd firebase && npm run seed          # demo officer, students, drives, applications
./gradlew installDebug               # live project when app/google-services.json exists
./gradlew testDebugUnitTest          # JVM unit tests
./gradlew connectedDebugAndroidTest  # instrumented tests (device/emulator)
cd firebase && npm test              # Firestore Security Rules tests
```

B. Source File Inventory

Package	Files
com.nexora	NexoraApp
com.nexora.domain (8)	Models, EligibilityEngine, MatchScoreCalculator, ReadinessAnalyzer, CareerActionEngine, ApplicationWorkflow, Analytics, Validators
com.nexora.data (8)	AuthRepository, PlacementRepository, ReadinessRepository, FirestoreMapper, LocalDb, SyncWorker, MessagingService, Resource
com.nexora.ui (11)	AuthActivity, MainActivity, StudentScreens, DriveScreens, ProfileScreens, OfficerScreens, DriveAdminScreens, StudentViewModel, OfficerViewModel, UiKit, ChartView
firebase/	firestore.rules, firestore.indexes.json, firebase.json, tests/rules.test.js, seed/seed.js

Table 7: Source files

C. Configuration and Security Notes

google-services.json, service-account keys and local.properties are excluded from version control. With google-services.json in app/, builds use the live project; without it, debug builds need no credentials because they use the Emulator Suite. Demo account passwords are defined in the seed script and are intentionally not reproduced in this report.